ПРАКТИЧЕСКАЯ РАБОТА №1

Тема: «Сравнение архитектуры операционных систем Windows NT, Linux и MINIX 3»

Дисциплина: …

Преподаватель: Семак Вячеслав Павлович

Выполнил: …

Группа: …

Дата выполнения: «__»_____ 20__ г.

2026

Оглавление

Введение....3
Цель и задачи работы....4
Средства выполнения работы....6
Ход работы....7
1. Рассматриваемые версии операционных систем....7
2. Архитектура Windows NT....8
3. Архитектура ОС на базе Linux....10
4. Архитектура MINIX 3....12
5. Сравнительный анализ архитектур....13
6. Влияние размещения компонентов на изоляцию, устойчивость и безопасность....15
7. Разбор ситуации компрометации драйвера....16
Вывод....17
Список использованных источников....18

Введение

Ядро операционной системы (ОС) выполняется в привилегированном режиме процессора и имеет полный доступ к оборудованию. Архитектура ОС определяет, какие компоненты работают в пространстве ядра, а какие вынесены в пространство пользователя. Различают монолитные ядра, микроядра и гибридные ядра, сочетающие оба подхода.

От архитектуры зависят производительность, устойчивость к отказам и информационная безопасность: ошибка в компоненте ядра способна нарушить работу всей системы, а сбой изолированного компонента затрагивает только его самого. В работе рассматриваются Windows NT (гибридное ядро), Linux (монолитное модульное ядро) и MINIX 3 (микроядро).

Цель и задачи работы

Цель работы:

Получение практических и теоретических навыков в сравнительном анализе архитектуры операционных систем. В данной практической работе необходимо произвести сравнение архитектуры Windows NT, Linux и MINIX 3, определить расположение основных системных компонентов и проанализировать значение архитектурных решений для изоляции, устойчивости и информационной безопасности.

Задачи для выполнения работы:

1. Изучить техническую документацию по архитектуре Windows NT, Linux и MINIX 3, указав рассматриваемые версии систем и использованные источники.
2. Определить тип ядра каждой операционной системы и выделить особенности её архитектурной организации.
3. Установить назначение и расположение основных компонентов ОС: ядра, системных служб, драйверов, компонентов файловой системы, средств управления процессами и памятью.
4. Рассмотреть способы взаимодействия приложений, системных служб, драйверов и ядра, обозначив границы пространства пользователя и пространства ядра.
5. Составить упрощённые архитектурные схемы Windows NT, Linux и MINIX 3, отразив размещение компонентов и связи между ними.
6. Выполнить сравнительный анализ трёх ОС по единым критериям и представить результаты в сравнительной таблице.
7. Проанализировать влияние размещения и полномочий системных компонентов на их изоляцию, устойчивость ОС и информационную безопасность.

8. Рассмотреть ситуацию компрометации драйвера, определить возможные последствия для конфиденциальности, целостности и доступности и предложить меры снижения риска.

9. Сформулировать обоснованные выводы о сходствах, различиях и ограничениях исследованных архитектур.

10. Оформить отчёт, содержащий архитектурные схемы, сравнительную таблицу, разбор ситуации, выводы и список источников.

Средства выполнения работы

Для выполнения работы были использованы следующие средства:

- технические: персональный компьютер с доступом к сети Интернет;
- программные: браузер, текстовый редактор, графический редактор для построения схем, средство просмотра PDF;
- информационные: документация Microsoft [1–4], документация ядра Linux и Ubuntu [5–7], материалы проекта MINIX 3 [8–10], учебная литература [11].

Ход работы

1. Рассматриваемые версии операционных систем

Для анализа были выбраны следующие версии ОС:

- Windows 11 версии 24H2 (ядро NT 10.0), x86-64 [1–4];
- Ubuntu 24.04 LTS с ядром Linux 6.8, x86-64 [5–7];
- MINIX 3.4.0, x86 [8–10].

Все три системы были рассмотрены по единому набору критериев, приведённому в сравнительной таблице (раздел 5).

2. Архитектура Windows NT

Ядро Windows NT относится к гибридному типу. Структура NT повторяет микроядерную модель (ядро, исполнительная система, подсистемы окружения), но ради производительности исполнительная система, драйверы и графическая подсистема Win32k.sys выполняются в режиме ядра в общем адресном пространстве [1, 2].

В режиме ядра находятся уровень аппаратных абстракций HAL, ядро (планирование потоков, прерывания), исполнительная система ntoskrnl.exe (диспетчеры объектов, ввода-вывода, памяти, процессов, монитор безопасности, реестр, ALPC) и драйверы. В пространстве пользователя работают приложения, системные процессы (smss, csrss, winlogon, lsass), службы, библиотеки подсистем и ntdll.dll, а также драйверы UMDF [3].

Приложение вызывает Win32 API, библиотека обращается к ntdll.dll, которая инструкцией syscall передаёт управление ядру; ввод-вывод выполняется пакетами IRP через стек драйверов (Рисунок 1 – Упрощённая архитектурная схема Windows NT (Windows 11, версия 24H2)).

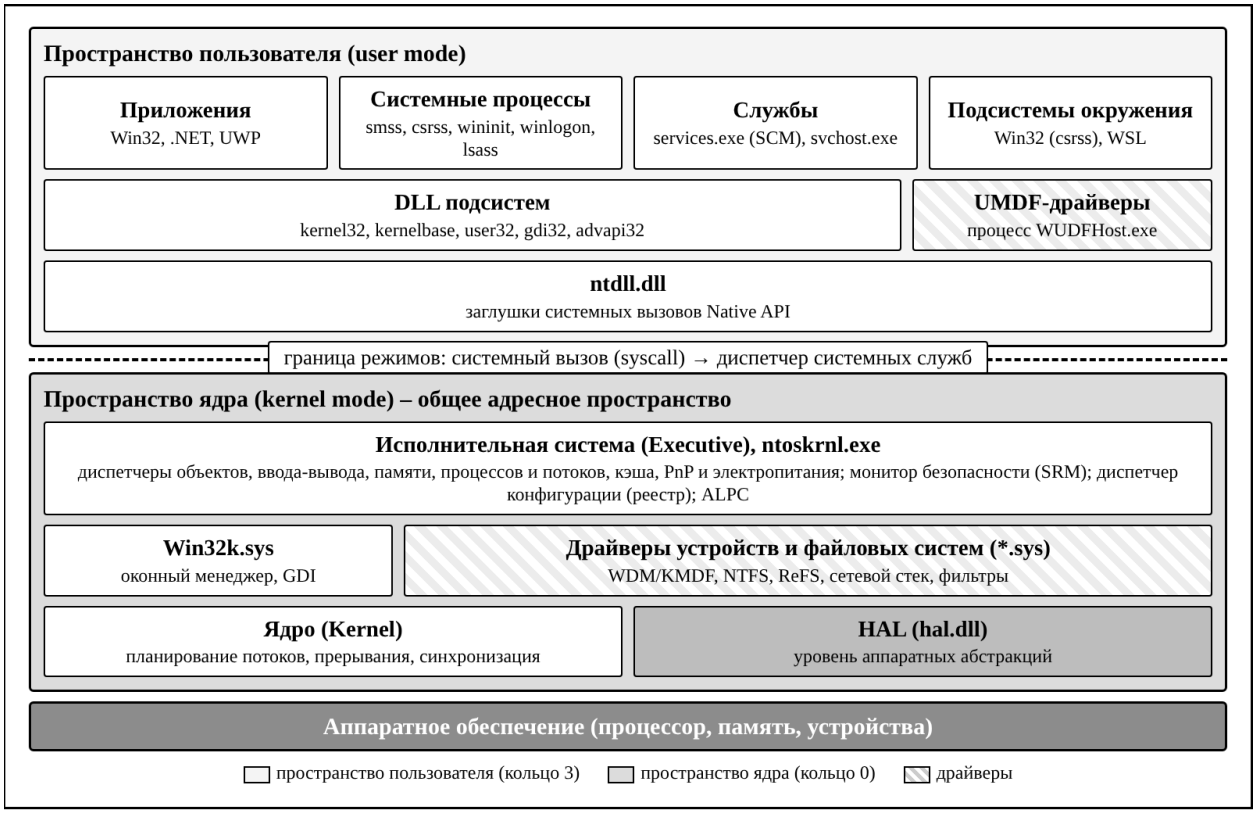


Рисунок 1 – Упрощённая архитектурная схема Windows NT (Windows 11, версия 24H2)

Компоненты режима ядра не изолированы друг от друга. Дополнительную изоляцию даёт технология VBS: гипервизор выделяет защищённую среду для Secure Kernel и процесса LsaIso.exe [1].

3. Архитектура ОС на базе Linux

Ядро Linux монолитное: планировщик, управление памятью, VFS, файловые системы, сетевой стек и драйверы работают в едином адресном пространстве и вызывают функции друг друга напрямую. Драйверы могут подключаться как загружаемые модули (*.ko), но после загрузки получают полные полномочия ядра [5].

В ядре 6.8 используется планировщик EEVDF, страничная виртуальная память с копированием при записи, cgroups и namespaces для изоляции групп процессов, модули безопасности LSM (AppArmor в Ubuntu). В пространстве пользователя работают приложения, glibc, systemd и системные службы; отдельные драйверы и ФС могут выполняться там же через FUSE, UIO и VFIO [5, 7].

Взаимодействие с ядром выполняется через системные вызовы, /proc, /sys, ioctl и netlink; всё, что расположено ниже этой границы, включая сторонние модули, работает в нулевом кольце защиты (Рисунок 2 – Упрощённая архитектурная схема ОС на базе Linux (Ubuntu 24.04 LTS, ядро 6.8)).

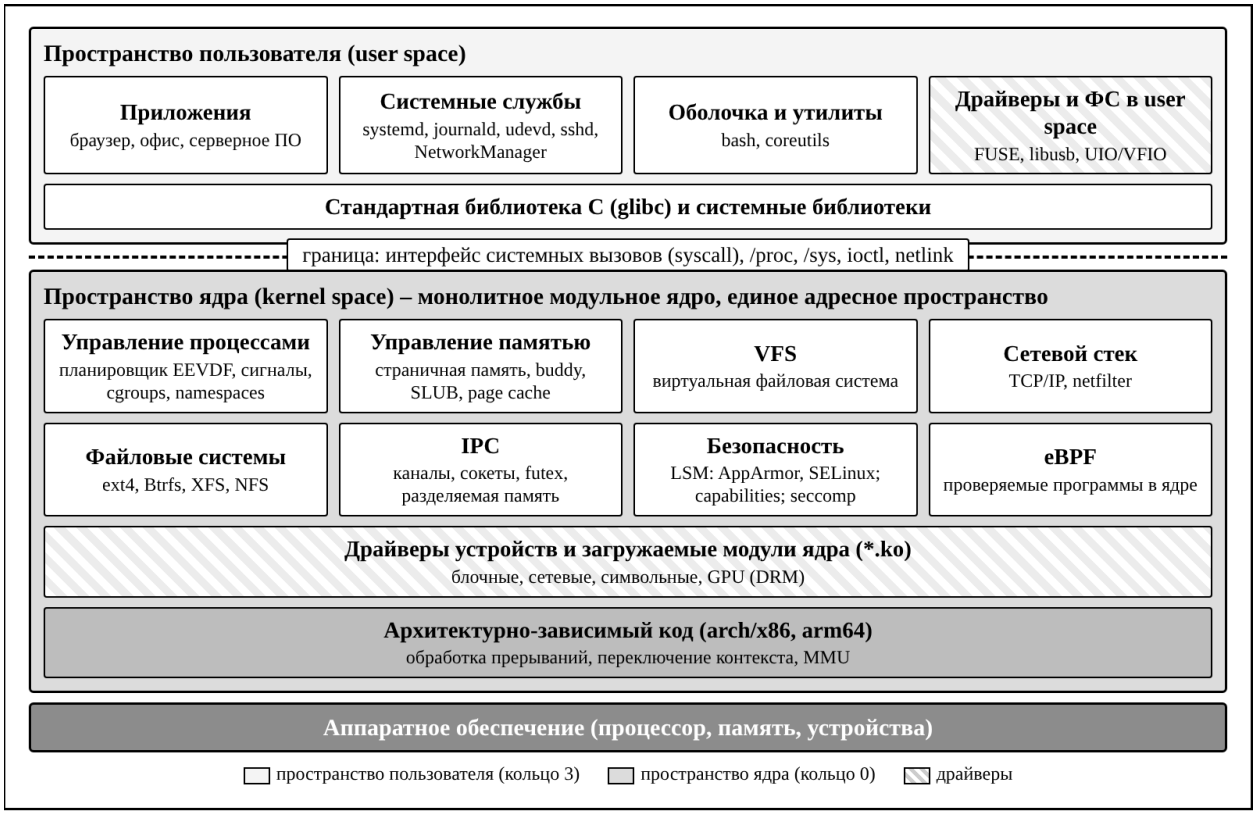


Рисунок 2 – Упрощённая архитектурная схема ОС на базе Linux (Ubuntu 24.04 LTS, ядро 6.8)

4. Архитектура MINIX 3

MINIX 3 построена на микроядре объёмом несколько тысяч строк кода, которое обрабатывает прерывания, переключает процессы и передаёт сообщения. В ядре также работают задачи SYSTEM и CLOCK [8, 9].

Остальные компоненты – отдельные процессы пространства пользователя: драйверы устройств, серверы PM (процессы), VM (память), SCHED (планирование), VFS и MFS (файловые системы), INET (сеть), DS (хранилище данных), RS (сервер реинкарнации) и пользовательские процессы [8, 10].

Компоненты взаимодействуют только сообщениями через ядро (send, receive, sendrec, notify). Права процесса ограничены маской привилегий, а доступ к чужой памяти возможен только через гранты (Рисунок 3 – Упрощённая архитектурная схема MINIX 3 (версия 3.4.0)).

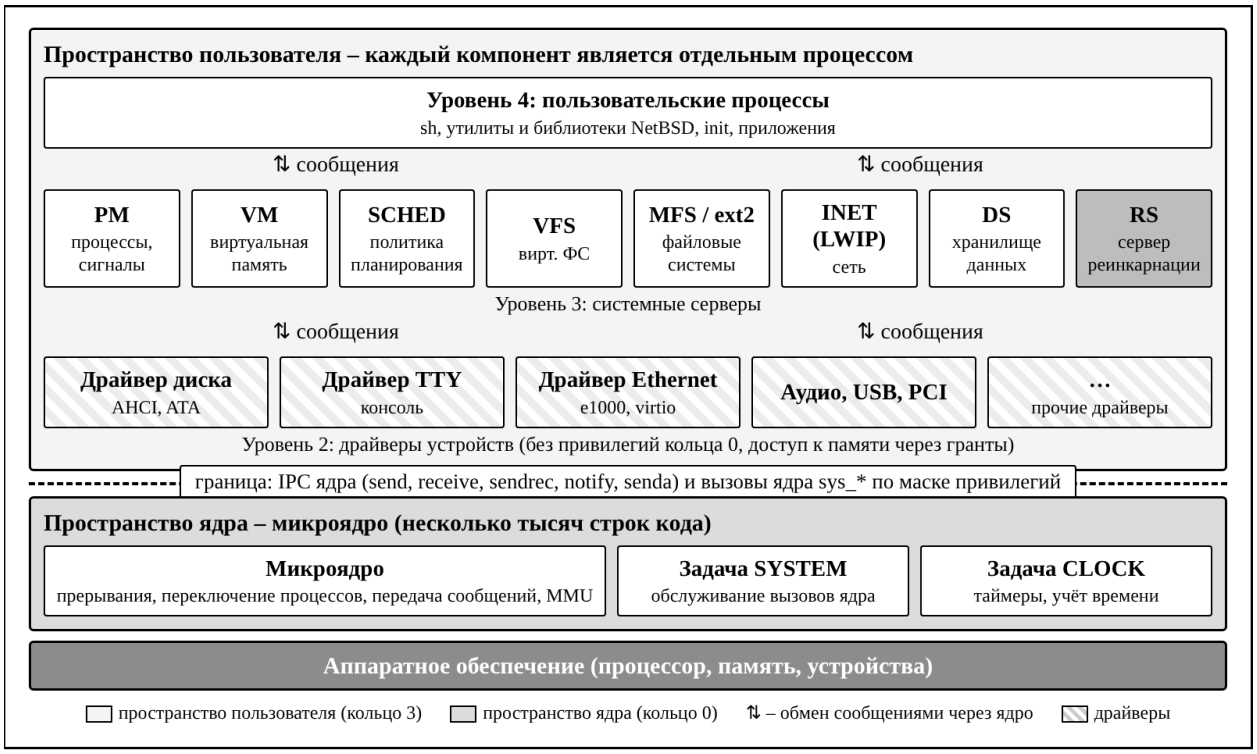


Рисунок 3 – Упрощённая архитектурная схема MINIX 3 (версия 3.4.0)

Сервер реинкарнации RS контролирует драйверы и серверы и автоматически перезапускает отказавший компонент; для блочных драйверов восстановление прозрачно для приложений [9].

5. Сравнительный анализ архитектур

Результаты сравнения трёх ОС по единым критериям сведены в таблицу (Таблица 1 – Сравнительная таблица операционных систем).

Таблица 1 – Сравнительная таблица операционных систем

Критерий	Windows NT	ОС на базе Linux	MINIX 3
Рассматриваемая версия ОС	Windows 11 24H2 (NT 10.0)	Ubuntu 24.04 LTS, ядро 6.8	MINIX 3.4.0
Название и тип ядра	ntoskrnl.exe; гибридное	Linux; монолитное модульное	Микроядро MINIX 3
Основные компоненты пространства ядра	HAL, ядро, исполнительная система, Win32k.sys, драйверы и ФС	Планировщик, память, VFS и ФС, сетевой стек, LSM, драйверы и модули	Микроядро, задачи SYSTEM и CLOCK
Основные компоненты пространства пользователя	Приложения, системные процессы, службы, DLL подсистем, ntdll.dll, UMDF	Приложения, glibc, systemd и службы, FUSE	Приложения, серверы PM, VM, VFS, RS и др., все драйверы
Взаимодействие приложений, служб и ядра	Win32 API → ntdll.dll → syscall; ALPC, RPC; IRP	Системные вызовы, /proc, /sys, ioctl, netlink	Сообщения через ядро; вызовы ядра по маске привилегий
Размещение драйверов устройств	В ядре (WDM, KMDF); часть – UMDF	В ядре (модули *.ko); часть – FUSE, VFIO	В пространстве пользователя, каждый – отдельный процесс
Размещение файловых служб	Драйверы ФС (ntfs.sys) в ядре	VFS и ФС (ext4) в ядре	Серверы VFS и MFS в пространстве пользователя
Управление процессами и памятью – основные особенности	Приоритетное вытесняющее планирование потоков; страничная память	Планировщик EEVDF, cgroups; страничная память, копирование при записи	Механизм в ядре, политика в серверах PM, SCHED, VM
Способы изоляции системных компонентов	Кольца 0 и 3, адресные пространства процессов, VBS; внутри ядра изоляции нет	Кольца 0 и 3, namespaces, seccomp, LSM; внутри ядра изоляции нет	Отдельное адресное пространство и маска привилегий у каждого компонента

Критерий	Windows NT	ОС на базе Linux	MINIX 3
Возможные последствия сбоя драйвера	Критическая ошибка («синий экран»), перезагрузка	Oops или kernel panic	Завершается только процесс драйвера
Средства восстановления после отказа компонента	Автоперезагрузка, дампы памяти, WinRE, перезапуск служб	kdump, panic=N, перезапуск служб systemd	Автоматический перезапуск компонента сервером RS
Архитектурные особенности, значимые для безопасности	Подпись драйверов, PatchGuard, HVCI; большая TCB	Подпись модулей, lockdown, KASLR; большая TCB	Минимальная TCB, наименьшие привилегии; ниже производительность

Конец Таблицы 1

Во всех трёх системах используются разделение режимов процессора и изоляция адресных пространств процессов. Главное различие – объём кода с максимальными привилегиями: в Windows NT и Linux драйверы, файловые системы и сетевой стек находятся в ядре, в MINIX 3 – в отдельных непривилегированных процессах.

6. Влияние размещения компонентов на изоляцию, устойчивость и безопасность

Компонент в пространстве ядра имеет доступ ко всей памяти и оборудованию, и последствия его ошибки ничем не ограничены. Поэтому архитектура влияет на систему следующим образом:

- изоляция: в Windows NT и Linux граница проходит только между пользователем и ядром, в MINIX 3 – вокруг каждого компонента;
- устойчивость: отказ драйвера в Windows NT и Linux обычно приводит к отказу всей системы, в MINIX 3 – устраняется перезапуском драйвера;
- безопасность: уязвимость в любом драйвере Windows или модуле Linux даёт привилегии ядра;
- производительность: общее адресное пространство избавляет от накладных расходов на сообщения, поэтому Windows NT и Linux быстрее MINIX 3.

Windows и Linux компенсируют эти недостатки подписью драйверов и модулей, HVCI, lockdown и KASLR [4, 6], а также выносом части драйверов в пространство пользователя (UMDF, FUSE).

7. Разбор ситуации компрометации драйвера

Рассмотрена ситуация, в которой нарушитель с правами администратора загружает подписанный, но уязвимый драйвер (атака BYOVD) и выполняет произвольный код с его полномочиями. Последствия приведены в таблице (Таблица 2 – Последствия компрометации драйвера для конфиденциальности, целостности и доступности).

Таблица 2 – Последствия компрометации драйвера для конфиденциальности, целостности и доступности

Свойство информации	Windows NT	ОС на базе Linux	MINIX 3
Конфиденциальность	Чтение памяти ядра и процессов, включая учётные данные lsass.exe	Чтение памяти любых процессов и ключей шифрования	Доступ только к данным самого драйвера
Целостность	Изменение структур ядра, отключение защиты, руткит	Изменение структур ядра, отключение LSM и аудита	Искажение данных драйвера; ядро и другие компоненты защищены
Доступность	Отказ всей системы	Kernel panic, отказ всей системы	Отказ одного устройства до перезапуска драйвера

В Windows NT и Linux компрометация драйвера равнозначна компрометации всего узла, в MINIX 3 ущерб ограничен одним драйвером. Для снижения риска предлагаются следующие меры:

1. допускать только подписанные драйверы и блокировать известные уязвимые драйверы [4, 6];
2. включить Secure Boot, а в Windows – VBS и HVCI;
3. удалить неиспользуемые драйверы и модули, применять IOMMU;
4. своевременно устанавливать обновления и ограничить круг администраторов;
5. контролировать загрузку драйверов и модулей средствами аудита.

Вывод

В ходе выполнения практической работы были получены практические и теоретические навыки в сравнительном анализе архитектуры операционных систем Windows NT, Linux и MINIX 3.

Было установлено, что Windows NT использует гибридное ядро, Linux – монолитное модульное, MINIX 3 – микроядро. Общими являются разделение режимов процессора и изоляция процессов, различие – в объёме кода, выполняемого в ядре. В Windows NT и Linux компрометация драйвера ведёт к компрометации всего узла, в MINIX 3 – ограничена одним драйвером. Монолитные и гибридные ядра выигрывают в производительности, но требуют дополнительных мер защиты, а микроядро обеспечивает лучшую изоляцию ценой скорости работы.

Список использованных источников

1. Yosifovich P., Ionescu A., Russinovich M. E., Solomon D. A. Windows Internals. Part 1. – 7th ed. – Redmond : Microsoft Press, 2017. – 800 p.
2. Overview of Windows Components [Электронный ресурс] // Microsoft Learn. – URL: https://learn.microsoft.com/en-us/windows-hardware/drivers/kernel/overview-of-windows-components (дата обращения: 09.11.2026).
3. User mode and kernel mode [Электронный ресурс] // Microsoft Learn. – URL: https://learn.microsoft.com/en-us/windows-hardware/drivers/gettingstarted/user-mode-and-kernel-mode (дата обращения: 09.11.2026).
4. Microsoft recommended driver block rules [Электронный ресурс] // Microsoft Learn. – URL: https://learn.microsoft.com/en-us/windows/security/application-security/application-control/app-control-for-business/design/microsoft-recommended-driver-block-rules (дата обращения: 09.11.2026).
5. The Linux Kernel documentation [Электронный ресурс]. – URL: https://docs.kernel.org/ (дата обращения: 09.11.2026).
6. Kernel module signing facility [Электронный ресурс] // The Linux Kernel documentation. – URL: https://docs.kernel.org/admin-guide/module-signing.html (дата обращения: 09.11.2026).
7. Ubuntu 24.04 LTS (Noble Numbat) release notes [Электронный ресурс] // Ubuntu Community Hub. – URL: https://discourse.ubuntu.com/t/ubuntu-24-04-lts-noble-numbat-release-notes/39890 (дата обращения: 09.11.2026).
8. Tanenbaum A. S., Woodhull A. S. Operating Systems: Design and Implementation. – 3rd ed. – Upper Saddle River : Prentice Hall, 2006. – 1054 p.
9. Herder J. N., Bos H., Gras B., Homburg P., Tanenbaum A. S. MINIX 3: A Highly Reliable, Self-Repairing Operating System // ACM SIGOPS Operating Systems Review. – 2006. – Vol. 40, No. 3. – P. 80–89.

10. MINIX 3 [Электронный ресурс] : официальный сайт проекта. – URL: https://www.minix3.org/ (дата обращения: 09.11.2026).

11. Таненбаум Э., Бос Х. Современные операционные системы. – 4-е изд. – СПб. : Питер, 2015. – 1120 с.